# PlaneWeave: Canonicalizing Bounding Planes for Consistent Video Editing - Supplementary Material -

Chaeeun Moon, Jongsung Lee, Seungoh Han, Kyungdon Joo*
UNIST, South Korea
{cemoon, syniez, sohan, kyungdon}@unist.ac.kr

This supplementary material consists of three parts. We first provide additional method details in Sec. A, followed by the data preparation details for object bounding-plane annotation in Sec. B. We then present additional experimental results in Sec. C.

## A. Additional Method Details

We provide additional method details that clarify how *PlaneWeave* maps video observations into the shared canonical editing domain. In particular, we describe the detailed geometric transformations used for canonicalization (Sec. A.1) and correspondence construction (Sec. A.2), followed by the triangulation procedure used to obtain canonical constraints (Sec. A.3), and the network architectures used for the final spatio-temporal canonicalization (Sec. A.4). Lastly, we detail the texture-map construction that maps edits into the canonical space from two editing domains: the source video frame and the canonical atlas (Sec. A.5).

### A.1. Geometric transformations

**Inputs and Coordinate Conventions.** Our method operates on RGB frames $\{\mathbf{I}_t\}_{t\in\mathcal{T}}$, object masks $\{\mathbf{M}_t\}_{t\in\mathcal{T}}$, camera intrinsics $\{\mathbf{K}_t\}_{t\in\mathcal{T}}$, camera pose in world coordinate system $\{(\mathbf{R}^{\mathrm{c}}_t, \mathbf{t}^{\mathrm{c}}_t)\}_{t\in\mathcal{T}}$, object pose in world coordinate system $\{(\mathbf{R}^{\mathrm{o}}_t, \mathbf{t}^{\mathrm{o}}_t)\}_{t\in\mathcal{T}}$ for time domain $\mathcal{T}$.

And for an object-centric set of bounding planes $\mathcal{B}$, we define each bounding plane $\mathcal{B}_i \in \mathcal{B}$'s time-independent relative pose as

$$\mathbf{T}^{o\leftarrow\mathcal{B}_i} = \begin{bmatrix} \mathbf{x}^{\mathrm{o}}_{\mathcal{B}_i} & \mathbf{y}^{\mathrm{o}}_{\mathcal{B}_i} & \mathbf{n}^{\mathrm{o}}_{\mathcal{B}_i} & \mathbf{o}^{\mathrm{o}}_{\mathcal{B}_i} \\ 0 & 0 & 0 & 1 \end{bmatrix}, \tag{1}$$

where $\mathbf{x}^{\mathrm{o}}_{\mathcal{B}_i}$, $\mathbf{y}^{\mathrm{o}}_{\mathcal{B}_i}$, $\mathbf{n}^{\mathrm{o}}_{\mathcal{B}_i}$, $\mathbf{o}^{\mathrm{o}}_{\mathcal{B}_i}$ are $x$-axis, $y$-axis, $z$-axis and a plane center of $\mathcal{B}_i$ in the object-centric coordinate system.

We perform weaving operations the object-centric coordinate system. Thus, object motion is removed before constructing the canonical representation, while the camera and object poses are used only to transform image observations into object-centric coordinate rays.

**Object-centric Bounding-Plane Projection.** For an image pixel $\mathbf{x} = (u, v)$ at frame $t$, we first construct its world-space ray using the camera intrinsics,

$$\begin{aligned} \mathbf{o}^{\mathrm{w}}_t &= \mathbf{t}^{\mathrm{c}}_t, \\ \bar{\mathbf{d}}^{\mathrm{w}}_t = \mathbf{R}^{\mathrm{c}}_t \mathbf{K}_t^{-1} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}, \quad \mathbf{d}^{\mathrm{w}}_t &= \frac{\bar{\mathbf{d}}^{\mathrm{w}}_t}{\|\bar{\mathbf{d}}^{\mathrm{w}}_t\|_2}, \end{aligned} \tag{2}$$

where $\mathbf{o}^{\mathrm{w}}_t$ and $\mathbf{d}^{\mathrm{w}}_t$ are ray origin and direction corresponding to $\mathbf{x}$. We then transform them into an object coordinate system, obtaining:

$$\begin{aligned} \mathbf{o}^{\mathrm{o}}_t &= (\mathbf{R}^{\mathrm{o}}_t)^{-1}(\mathbf{o}^{\mathrm{w}}_t - \mathbf{t}^{\mathrm{o}}_t), \\ \mathbf{d}^{\mathrm{o}}_t &= (\mathbf{R}^{\mathrm{o}}_t)^{-1}\mathbf{d}^{\mathrm{w}}_t. \end{aligned} \tag{3}$$

We then intersect the ray with each bounding plane $\mathcal{B}_i$ considering its polygonal boundary:

$$\begin{aligned} \mathbf{q}^{\mathrm{o}}_t = \mathbf{o}^{\mathrm{o}}_t + \lambda \mathbf{d}^{\mathrm{o}}_t,\ \lambda = \frac{(\mathbf{o}^{\mathrm{o}}_{\mathcal{B}_i} - \mathbf{o}^{\mathrm{o}}_t)\cdot \mathbf{n}^{\mathrm{o}}_{\mathcal{B}_i,t}}{\mathbf{d}^{\mathrm{o}}_t \cdot \mathbf{n}^{\mathrm{o}}\mathcal{B}_i, t}, \\ \mathbf{q}^{\mathrm{o}}_t \in \mathcal{B}_i, \end{aligned} \tag{4}$$

and retain the nearest valid positive intersection. We simply write the intersection $\mathbf{q}^{\mathrm{o}}_t$ as $\Pi^t_{\mathcal{B}}(\mathbf{x})$ in the main paper.

Then, importantly, we parametrize the ray intersection $\mathbf{q}^{\mathrm{o}}_t$ in object-centric coordinate system into $\mathbf{s}_t = \frac{\mathbf{q}^{\mathrm{o}}_t}{\|\mathbf{q}^{\mathrm{o}}_t\|_2}$ on a unit sphere $S^2$.

### A.2. Correspondence Construction

For establishing canonical targets by tracks, we select a reference frame $r_i$ for each bounding plane $\mathcal{B}_i$ based on its projected object-mask coverage. From $r_i$, we sample query pixels whose first bounding-surface intersections lie on $\mathcal{B}_i$. Then we aggregate those samples as track queries $\{\mathbf{x}^n\}$, where we denote the whole indices as $\mathcal{N}$. We propagate each query $\mathbf{x}^n$ throughout the video using AllTracker [1],

*Corresponding author.

which provides the tracked position $\mathbf{x}_t^n$, visibility $v_t^n$, and confidence $c_t^n$ at each frame $t$.

We filter the propagated observations using both tracking reliability and bounding-surface consistency. Let $i_t^n$ denote the index of the bounding plane containing the first surface intersection of $\mathbf{x}_t^n$. For a track initialized on $\mathcal{B}_i$, its valid observations are

$$\tilde{\mathcal{V}}^n = \left\{ t \;\middle|\; \begin{array}{c} v_t^n \geq 0.5,\ c_t^n \geq 0.5, \\ \mathbf{x}_t^n \in M_t,\ i_t^n \in \{i\} \cup N(i) \end{array} \right\}, \tag{5}$$

where $N(i)$ denotes the set of indices of bounding planes adjacent to $\mathcal{B}_i$. Allowing observations on adjacent planes accounts for changes in the first-hit bounding-surface projection near plane boundaries under viewpoint variation.

For each retained observation, we initially define its track reliability as

$$\tilde{w}_t^n = v_t^n c_t^n, \tag{6}$$

which is used to weight the object-centric-space rays during triangulation. We further modulate the weight $\tilde{w}_t^n$ to final anchor weight $w_t^n$ after geometric and reprojection validation in the triangulation procedure in the following section.

### A.3. Triangulation and Canonical Anchors

After getting correspondence tracks, we perform a triangulation in ray space not in image coordinate space, and use projection $\mathbf{P}_t^{\mathrm{o}}$ from the common object-centric coordinate system to frame $t$, incorporating the object pose, camera pose, and camera intrinsics for computing reprojection error for computing the final weight $w_t^n$.

**Triangulation from multi-view rays.** Specifically, for the track $n \in \mathcal{N}$ we triangulate the frame coordinate $\mathbf{x}_t^n$ in the valid frame $t \in \tilde{\mathcal{V}}^n$. For an observation $\mathbf{x}_t^n$, we can obtain the corresponding ray origin and direction $\mathbf{o}_t^n, \mathbf{d}_t^n$ in the object coordinate system by Eq. 3.

We estimate the 3D anchor $X^n$ by minimizing the viewing ray $\mathbf{X} - \mathbf{o}_t^n$'s component perpendicular to the ray direction, weighted by the track reliability as

$$\mathbf{X}^n = \arg\min_{\mathbf{X}} \sum_{t \in \mathcal{V}^n} \tilde{w}_t^n \left\| \left( \mathbf{I} - \mathbf{d}_t^n \mathbf{d}_t^{n\top} \right) (\mathbf{X} - \mathbf{o}_t^n) \right\|_2^2, \tag{7}$$

where $\tilde{w}_t^n = v_t^n c_t^n$. Here, the $\mathbf{d}_t^n \mathbf{d}_t^{n\top} (\mathbf{X} - \mathbf{o}_t^n) = \mathbf{d}_t^{n\top} (\mathbf{X} - \mathbf{o}_t^n) \mathbf{d}_t^n$ is the viewing ray's parallel component to the ray direction.

**Geometric and reprojection validation.** We robustify the estimate by iteratively removing observations whose angular inconsistency with the estimated 3D point exceeds $2^\circ$, as well as observations for which the estimated point has non-positive depth along the corresponding viewing ray. We require at least two valid observations and reject geometrically degenerate ray configurations. After the geometric filtering, we further validate the estimated point in image space using the object-to-image projection $\mathbf{P}_t^{\mathrm{o}}$. We compute the reprojection error for observation $t$ of track $n$ as

$$e_t^n = \left\| \mathbf{P}_t^{\mathrm{o}}(\mathbf{X}^n) - \mathbf{x}_t^n \right\|_2. \tag{8}$$

Observations with $e_t^n > 5$ pixels are removed, and the 3D point is refitted using the remaining observations. We denote the final surviving frame set by $\mathcal{V}^n$.

The resulting object-space point is converted to the canonical target on the unit sphere as

$$\mathbf{u}^n = \rho(\mathbf{X}^n) = \frac{\mathbf{X}^n}{\|\mathbf{X}^n\|_2} \in S^2. \tag{9}$$

Finally, we incorporate the reprojection quality of each track into its anchor reliability. We first define the track-level reprojection quality as

$$q^n = \exp\left( -\frac{\mathrm{median}_{t \in \mathcal{V}^n} e_t^n}{\tau_{\mathrm{repr}}} \right), \tag{10}$$

where $\tau_{\mathrm{repr}}$ is 5 pixels. We define the final reliability of each surviving observation of the track $n$ as

$$w_t^n = v_t^n c_t^n q^n, \qquad t \in \mathcal{V}^n. \tag{11}$$

The resulting tuples $(\mathbf{s}_t^n, \mathbf{u}^n, w_t^n)$ form sparse canonical constraints used to guide the following frame-wise spatial weave and the spatio-temporal weave.

### A.4. Canonicalization and Appearance Models

We provide the architectural details of the learned spatio-temporal canonicalization and appearance models described in the main paper.

**Spatio-temporal canonicalization.** As described in the main paper, the canonicalization mapping $W_\theta(\mathbf{s}, t)$ is obtained by accumulating the tangent velocity field $V_\theta(\mathbf{q}, t, v)$ over $K$ spherical integration steps. We use $K = 8$ for all experiments.

We parameterize $V_\theta$ using a coordinate-based MLP. The current spherical coordinate $\mathbf{q} \in S^2$ is encoded using degree-4 spherical harmonics, while the frame time $t$ and accumulation state $v$ are encoded using frequency encodings with 6 and 4 frequencies, respectively. The encoded features are concatenated and processed by an MLP with two hidden layers of 64 units and ReLU activations. The network predicts a 3D offset without an output activation, which is scaled by $0.05$ before being projected onto the tangent plane of $S^2$.

The frame-wise weave $F_t$ is used only to bootstrap the accumulated canonicalization, as described in the main paper, and is not composed with $W_\theta$ during inference.

**Canonical appearance model.** We represent the canonical atlas $A_\phi \colon S^2 \to [0,1]^3$ using a separate coordinate-based

MLP. The canonical spherical coordinate is encoded using degree-4 spherical harmonics and processed by an MLP with three hidden layers of 64 units and ReLU activations. A sigmoid output activation predicts the RGB atlas color. The appearance model is RGB-only and does not use an additional learned opacity or support head.

We implement both coordinate-based networks using the efficient neural network framework of Müller [3].

### A.5. Object Editing from the Canonicalized Atlas

Our editing interface supports textures specified either in a source video frame or directly in the canonical atlas domain. Regardless of the input representation, we first convert the edit into a set of samples defined in the shared canonical domain. These canonical samples can then be rasterized either into an equirectangular texture map or into a collection of bounding-plane texture maps, and subsequently queried to propagate the edit across frames.

**Edit canonicalization.** We represent the edit frame points as $\mathcal{E}_t$. For the source-frame editing, we can represent a set of canonical samples form the input edit as

$$\mathcal{E}^{\text{can}} = \{\big(Z(\mathbf{x}_i, t), \text{RGBA}_t(\mathbf{x}_i)\big) | \mathbf{x}_i \in \mathcal{E}_t\}, \tag{12}$$

where $Z$ is a mapping from the video coordinate to the spherical canonical atlas and RGBA denotes the associated $\text{RGBA}_t$ value at the edit frame coordinate respectively.

For an atlas-space editing, the user can specify an edit directly in the 2D equirectangular atlas. We denote the ERP edit frame points $\mathcal{E}_{\text{ERP}}$. Each normalized atlas coordinate $(u, v) \in [0, 1]^2$ is converted to its corresponding canonical direction $\mathbf{z}$ as

$$\begin{aligned} \varphi &= 2\pi \left(u - \frac{1}{2}\right), \\ \theta &= \pi \left(v - \frac{1}{2}\right), \\ \mathbf{z}(u, v) &= \begin{bmatrix} \cos\theta \cos\varphi \\ \cos\theta \sin\varphi \\ \sin\theta \end{bmatrix} \in S^2. \end{aligned} \tag{13}$$

This gives the following canonical samples

$$\mathcal{E}^{\text{can}} = \{\big(\mathbf{z}(u, v), \text{RGBA}_{\text{ERP}}(u, v)\big) | (u, v) \in \mathcal{E}_{\text{ERP}}\}, \tag{14}$$

where the $\text{RGBA}_{\text{ERP}}$ denotes the the associated RGBA value at the ERP edit frame coordinate.

In this way, both source-frame and atlas-space edits are converted into the same canonical representation before texture creation.

**2D texture map creation.** Given the canonical edit samples $\mathcal{E}^{\text{can}}$, we rasterize them into either an equirectangular texture map or a collection of per-plane texture maps. Both representations encode the same canonical edit and differ only in their 2D parameterization.

For equirectangular storage, each canonical coordinate $\mathbf{z} = [X, Y, Z]^\top \in S^2$ is mapped to normalized texture coordinates

$$\begin{aligned} u(\mathbf{z}) &= \frac{\text{atan2}(Y, X)}{2\pi} + \frac{1}{2}, \\ v(\mathbf{z}) &= \frac{\arcsin(Z)}{\pi} + \frac{1}{2}. \end{aligned} \tag{15}$$

The associated RGBA values are splatted onto their corresponding $(u, v)$ locations to construct the texture map $\mathbf{E}_{\text{ERP}}$ in the ERP domain.

Although the equirectangular representation provides a convenient single 2D atlas, its effective spatial resolution varies across the sphere and can introduce distortion, particularly near the poles. We therefore additionally support storing the canonical edit as a collection of texture maps defined over the canonical bounding planes.

For each canonical direction $\mathbf{z} \in S^2$, we cast an object-centric radial ray and determine its first intersecting bounding plane $\mathcal{B}_i$. Then we can compute the intersection point,

$$\mathbf{q}^{\text{o}}_{\mathcal{B}_i} = \lambda \mathbf{z}, \quad \lambda = \frac{\mathbf{o}^{\text{o}}_{\mathcal{B}_i} \cdot \mathbf{n}^{\text{o}}_{\mathcal{B}_i}}{\mathbf{z} \cdot \mathbf{n}^{\text{o}}_{\mathcal{B}_i}}. \tag{16}$$

We now map this intersection to the local texture map of the bounding plane as

$$\begin{aligned} \xi(\mathbf{z}) &= (\mathbf{q}^{\text{o}}_{\mathcal{B}_i} - \mathbf{o}^{\text{o}}_{\mathcal{B}_i}) \cdot \mathbf{x}^{\text{o}}_{\mathcal{B}_i}, \\ \eta(\mathbf{z}) &= (\mathbf{q}^{\text{o}}_{\mathcal{B}_i} - \mathbf{o}^{\text{o}}_{\mathcal{B}_i}) \cdot \mathbf{y}^{\text{o}}_{\mathcal{B}_i}. \end{aligned} \tag{17}$$

The corresponding RGBA value is then rasterized into the texture map $\mathbf{E}_{\mathcal{B}_i}$ for each bounding plane $\mathcal{B}_i$.

**Edit propagation.** To propagate the canonical edit to a target frame $t$, for each object pixel $\mathbf{x}$ we first compute its canonical coordinate

$$\mathbf{z}_t = W_\theta\left(\mathcal{S}(\mathbf{x}, t), t\right). \tag{18}$$

For the equirectangular texture map, we map $\mathbf{z}_t$ to its texture coordinate $(u(\mathbf{z}_t), v(\mathbf{z}_t))$ using Eq. (15) and directly query the texture map $E_{\text{ERP}}$. For the per-plane representation, we instead intersect the radial ray from the object center to $\mathbf{z}_t$ with the bounding planes, identify the intersecting plane $\mathcal{B}_i$, and query its texture map $E_{\mathcal{B}_i}$ at the corresponding local coordinates $(\xi(\mathbf{z}_t), \eta(\mathbf{z}_t))$. The resulting RGBA value is then alpha-composited with the original video frame.

## B. Object Bounding-Plane Annotation Pipeline

For the CO3Dv2 [4] sequences used in our experiments, including Cake, Toytruck, and Toaster, we construct the object-centric bounding planes from the provided sparse 3D point cloud. We use a manual annotation procedure to

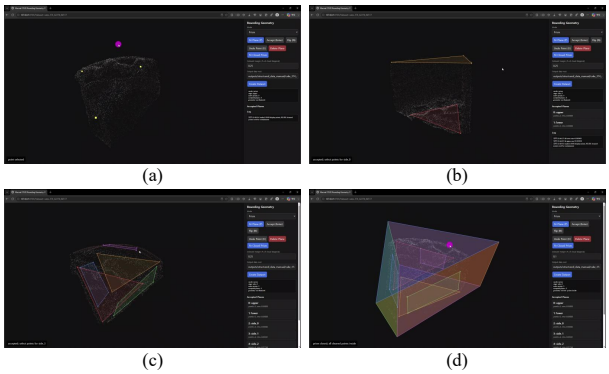


Figure 1. **Object bounding-plane annotation pipeline.** Annotation example for Cake CO3Dv2 subset. (a) Sparse 3D points selected from the CO3Dv2 point cloud for plane fitting. (b) Fitted upper and lower planes. (c) Fitted upper, lower, and side planes. (d) Final closed prism used to define the object-centric coordinate system and bounding-plane poses.

obtain approximate supporting planes and the final closed prism and its coordinate system are constructed automatically. Fig. 1 illustrates the bounding-plane annotation procedure.

We first robustly crop the CO3Dv2 point cloud and display a sampled subset for annotation. To specify a plane, the annotator selects at least three points approximately lying on the corresponding object surface. We fit a plane to these points using SVD, obtaining a plane origin and normal. The normal is oriented outward using the point-cloud center as a temporary reference.

For the prism geometry, we first annotate the upper and lower faces, followed by the side faces. The upper and lower planes determine a common prism axis. The side-plane normals are projected onto the plane orthogonal to this axis, and their intersections determine the polygonal cross-section of the prism. The plane offsets are expanded when necessary so that the final prism encloses the cleaned sparse point cloud with a small margin.

We accept the annotation only when the resulting planes form a valid closed surface: every side plane must contribute one edge to the cross-section, every prism edge must be shared by exactly two faces, all face normals must point outward, and all cleaned point-cloud points must lie inside the resulting prism.

The completed prism additionally defines a fixed object-centric coordinate system for the CO3Dv2 sequence. Since the objects in these sequences are treated as static in the world coordinate system, we use a single object-to-world transformation $\mathbf{T}^{w\leftarrow o}$ for all frames. Its origin $\mathbf{o}_o^w$ is set to the center of the final prism, its $z$-axis $\mathbf{z}_o^w$ is the common upper–lower prism axis, and the remaining axes form a right-handed coordinate frame $\mathbf{x}_o^w$, $\mathbf{y}_o^w$. Accordingly, we express the object pose as

$$\mathbf{T}^{w\leftarrow o} = \begin{bmatrix} \mathbf{x}_o^w & \mathbf{y}_o^w & \mathbf{z}_o^w & \mathbf{o}_o^w \\ 0 & 0 & 0 & 1 \end{bmatrix}. \tag{19}$$

This same object pose is associated with every frame of the sequence, while the changing viewpoint is represented by the frame-dependent camera poses.

For each final prism face $\mathcal{B}_i$, we obtain the bounding-plane pose $\mathbf{T}^{o\leftarrow\mathcal{B}_i}$ from the annotation. We construct its plane-to-object transformation using the face center $\mathbf{o}_{\mathcal{B}_i}^w$, two in-plane axes $\mathbf{x}_{\mathcal{B}_i}^w, \mathbf{y}_{\mathcal{B}_i}^w$, and outward normal $\mathbf{n}_{\mathcal{B}_i}^w$ as

$$\mathbf{T}^{w\leftarrow\mathcal{B}_i} = \begin{bmatrix} \mathbf{x}_{\mathcal{B}_i}^w & \mathbf{y}_{\mathcal{B}_i}^w & \mathbf{n}_{\mathcal{B}_i}^w & \mathbf{o}_{\mathcal{B}_i}^w \\ 0 & 0 & 0 & 1 \end{bmatrix}. \tag{20}$$

We store the plane poses together with their plane boundary and plane adjacency, yielding the object-centric bounding geometry used throughout our method.

For the Waymo [7] sequence, we use the provided object poses and convert the bounding boxes into a bounding-plane representation, as done for CO3Dv2, since a box is a special case of a prism.

## C. Additional Results

In this section, we provide additional qualitative results for comparison (Sec. C.1) and ablation (Sec. C.2) experiments in the main paper.

### C.1. Robustness to Viewpoint Variation

The main paper evaluates LNA [2], NAG [5], and our method on sequences covering approximately $180°$ of viewpoint variation. Here, we additionally evaluate progressively increasing rotation ranges of approximately $30°$, $90°$, and $180°$. The $30°$ setting represents a relatively limited-view case that LNA and NAG has mostly dealt with, while the $90°$ and $180°$ settings progressively expose larger portions of the object surface.

For each setting, all methods are independently trained on the same temporal subset, and we apply the same source-frame edit and propagate it across different viewpoints. Figure 2, 3, 4 shows the results across the three rotation ranges for Cake, Toytruck, Toaster sequences respectively.

While the methods are relatively comparable under limited viewpoint variation, their differences become more apparent as the rotation range increases. Our method maintains more consistent edit placement and appearance across larger viewpoint changes, suggesting that the object-enclosing bounding surface and learned canonical weave are particularly beneficial when substantially different object views must be represented in a shared editing domain.

| Dataset | Variant | Mean EPE ↓ | Median EPE ↓ | P95 EPE ↓ | PCK@5 ↑ | PCK@10 ↑ |
|---|---|---|---|---|---|---|
| Cake | *Projection* | 65.29 | 64.94 | 120.31 | 2.10 | 5.76 |
| | *Frame-wise* | **13.48** | **8.76** | *40.95* | **29.17** | **56.08** |
| | *Residual* | 13.82 | 9.14 | 40.47 | 26.42 | 54.36 |
| | *Full* | *14.36* | *10.09* | **33.21** | *24.02* | *49.59* |
| Toytruck | *Projection* | 63.04 | 42.44 | 158.93 | 4.32 | 10.29 |
| | *Frame-wise* | **9.08** | **3.84** | 32.55 | **55.27** | 68.94 |
| | *Residual* | 9.09 | 5.78 | **29.27** | 43.95 | **69.78** |
| | *Full* | *23.78* | *14.53* | *66.96* | *20.20* | *42.60* |
| Toaster | *Projection* | 72.04 | 66.35 | 151.56 | 3.98 | 5.83 |
| | *Frame-wise* | **18.73** | **14.48** | **47.11** | **14.84** | **34.05** |
| | *Residual* | 23.89 | 18.85 | 58.97 | 12.22 | 27.71 |
| | *Full* | *29.06* | *22.50* | *75.89* | *11.23* | *22.46* |
| Average | *Projection* | 66.79 | 57.91 | 143.60 | 3.47 | 7.29 |
| | *Frame-wise* | **13.76** | **9.03** | **40.20** | **33.09** | **53.03** |
| | *Residual* | 15.60 | 11.26 | 42.90 | 27.53 | 50.62 |
| | *Full* | *22.40* | *15.71* | *58.68* | *18.48* | *38.22* |

Table 1. Held-out correspondence accuracy of the canonicalization variants. EPE is reported in pixels and PCK in percentages. Best, second-best, and third-best results are shown in **bold**, underline, and *italics*, respectively.

| Dataset | Variant | ARAP ↓ | Stretch ↓ | Anisotropy ↓ | Area Dist. ↓ | Folds ↓ |
|---|---|---|---|---|---|---|
| Cake | *Projection*† | $7.74\times10^{-9}$ | $4.78\times10^{-5}$ | $4.49\times10^{-5}$ | $8.32\times10^{-5}$ | 0/188416 |
| | *Frame-wise* | 0.691 | *0.431* | 0.333 | *0.510* | *392 / 188416* |
| | *Residual* | *0.710* | 0.430 | *0.346* | 0.497 | 391 / 188416 |
| | *Full* | **0.133** | **0.222** | **0.213** | **0.299** | **0 / 188416** |
| Toytruck | *Projection*† | $1.41\times10^{-8}$ | $6.12\times10^{-5}$ | $5.56\times10^{-5}$ | $1.06\times10^{-4}$ | 0/169984 |
| | *Frame-wise* | 1.196 | 0.553 | 0.495 | 0.598 | 679 / 169984 |
| | *Residual* | *1.333* | *0.567* | *0.505* | *0.601* | *680 / 169984* |
| | *Full* | **0.313** | **0.332** | **0.274** | **0.439** | **0 / 169984** |
| Toaster | *Projection*† | $7.33\times10^{-9}$ | $5.16\times10^{-5}$ | $4.13\times10^{-5}$ | $9.32\times10^{-5}$ | 0/169979 |
| | *Frame-wise* | 1.946 | 0.658 | 0.447 | *0.711* | 1191 / 169979 |
| | *Residual* | *2.454* | *0.688* | *0.605* | 0.689 | *1360 / 169979* |
| | *Full* | **0.343** | **0.345** | **0.353** | **0.410** | **0 / 169979** |
| Average | *Projection*† | $9.72\times10^{-9}$ | $5.35\times10^{-5}$ | $4.73\times10^{-5}$ | $9.42\times10^{-5}$ | 0/528379 |
| | *Frame-wise* | 1.278 | 0.547 | 0.425 | *0.606* | 2262 / 528379 |
| | *Residual* | *1.499* | *0.562* | *0.485* | 0.596 | *2431 / 528379* |
| | *Full* | **0.263** | **0.300** | **0.280** | **0.383** | **0 / 528379** |

Table 2. Local deformation consistency of the canonicalization variants. Lower is better; folds are reported as folded neighborhoods over all evaluated neighborhoods. Best, second-best, and third-best results are shown in **bold**, underline, and *italics*, respectively, among learned variants. †*Projection* has near-zero distortion by construction and is included only as a reference.

## C.2. Additional Canonicalization Ablation Analysis

**Ablation setup.** We evaluate the three main variants—*Projection*, *Frame-wise*, and *Full*—and additionally consider a *Residual* variant for further analysis. The *Projection* variant uses the radial bounding planes projection without a learned canonical warp, while the *Frame-wise* variant uses the geometry-guided frame-wise canonicalization $F_t$. The *Residual* variant applies a learned spatio-temporal residual warp on top of fixed $F_t$, whereas the *Full* model uses $F_t$ to initialize the accumulated spatio-temporal canonicalization $W_\theta$ and employs the complete training objective. Unless otherwise specified, all variants use the same training data and optimization settings.

**Canonical correspondence consistency.** We evaluate canonical correspondence using held-out tracking correspondences in Table 1. Here, the values differ from those reported in Table 1 of the main paper because we evaluate the full sequence for each dataset. In the main paper, we instead use approximately $180°$ subsequences to enable a fairer comparison with plane-based methods, which are primarily designed for scenes with relatively limited viewpoint variation. The geometry-guided *Frame-wise* model achieves the lowest correspondence error across the evaluated sequences. The *Residual* variant retains comparable correspondence accuracy, while the *Full* model exhibits a larger deviation from the held-out point correspondences.

**Local deformation consistency.** Table 2 evaluates local geometric distortion using ARAP [6], stretch, anisotropy, area distortion, and folding. The *Residual* variant provides little improvement over *Frame-wise*, whereas the *Full* model consistently reduces local distortion and eliminates foldings across all sequences. The *Projection* variant is included only as a reference, as its near-zero distortion follows from having no learned canonical deformation.

**Correspondence–deformation trade-off.** Together, the two evaluations reveal complementary behavior of the canonicalization strategies. The geometry-guided *Frame-wise* model provides the most accurate pointwise correspondence, while the accumulated spatio-temporal canonicalization of the *Full* model produces substantially more regular local deformations. This trade-off is also reflected in the qualitative editing results in Fig. 5, where the *Full* model provides more stable edit propagation across viewpoints.

**Qualitative comparison.** Figure 5 provides additional qualitative comparisons of the *Projection*, *Frame-wise*, *Residual*, and *Full* variants on the Cake, Toytruck, and Toaster sequences. Consistent with the ablation in the main paper, direct bounding-plane projection produces noticeable variation in the location and shape of the propagated edit across viewpoints. The *Frame-wise* canonicalization substantially improves object-centric alignment, while the *Full* model further reduces local variation and produces more stable edit propagation across different object geometries and appearances.

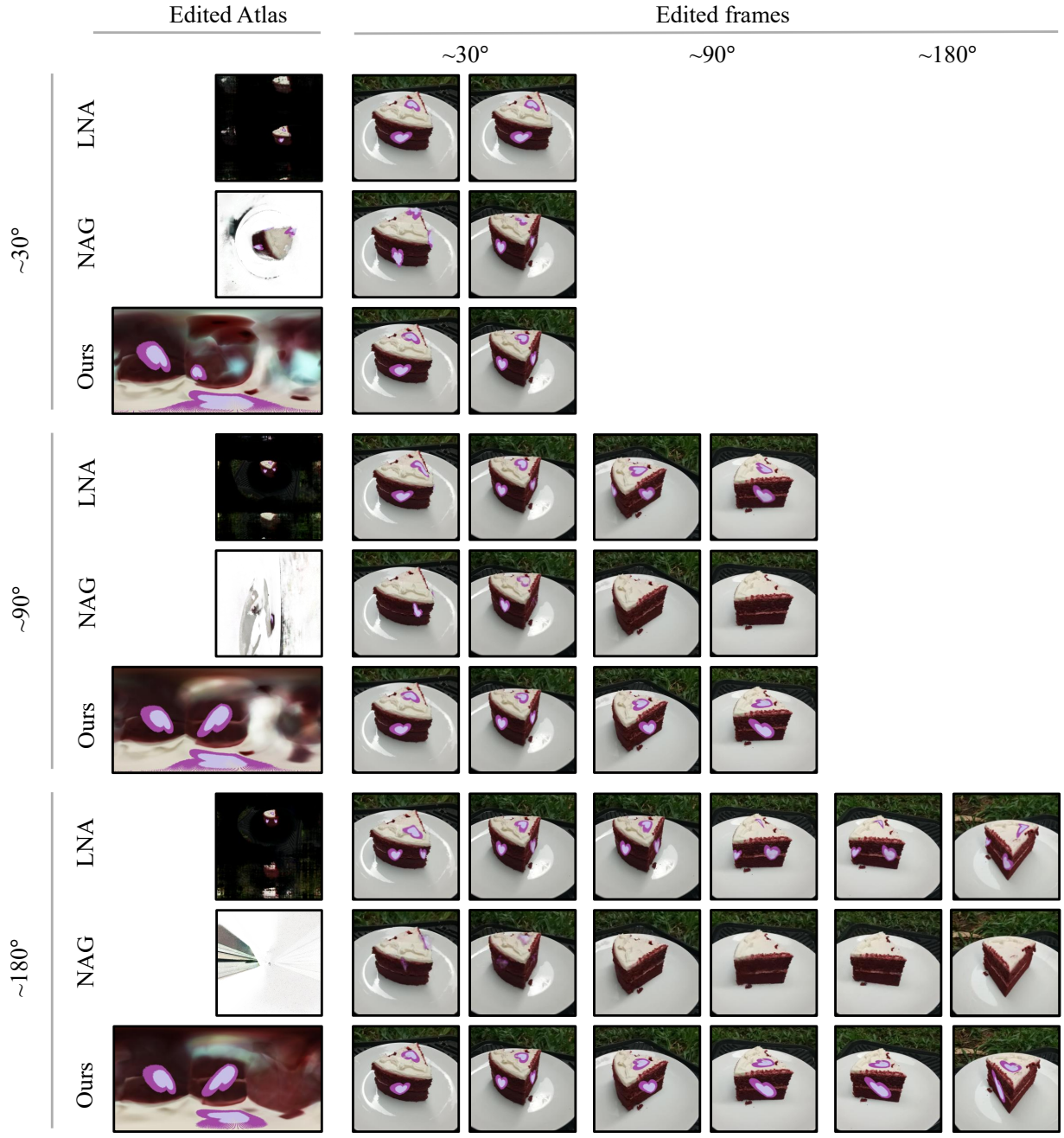



Figure 2. **Qualitative comparison of consistent appearance editing and canonical atlas reconstruction on the** Cake **sequence.** The 30°, 90°, and 180° labels on the left indicate the rotation range of the input subsequence used to reconstruct the atlas for each of the three compared methods. On the right, we show the reconstructed canonical atlas with the same input texture in their canonical domain, together with selected edited frames sampled across the corresponding rotation range.

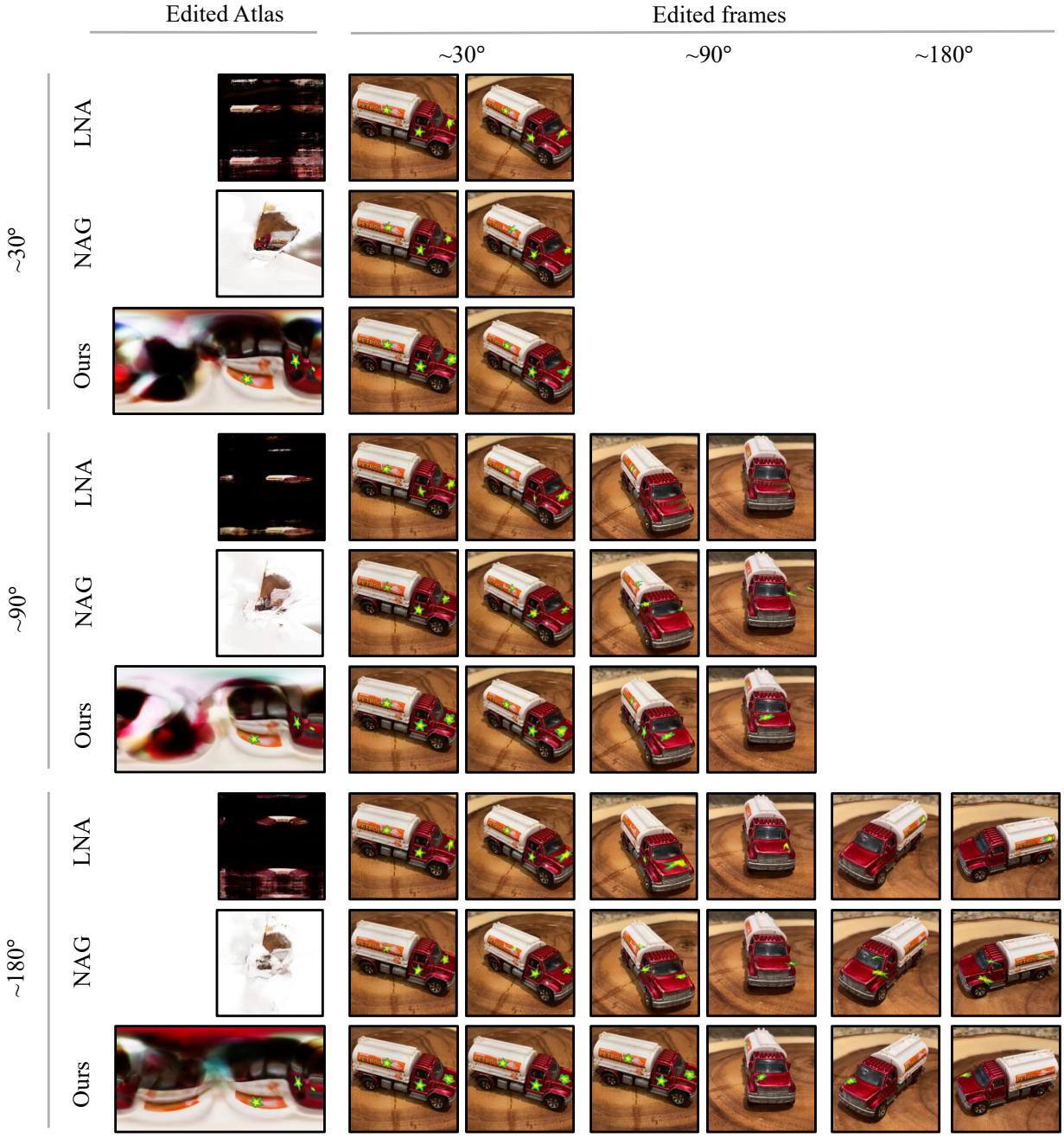



Figure 3. **Qualitative comparison on the** Toytruck **sequence.** Following the same visualization protocol as Fig. 2, we compare canonical atlas reconstruction and consistent appearance editing using input subsequences covering 30°, 90°, and 180° of object rotation. The reconstructed canonical atlases and selected edited frames are shown on the right.

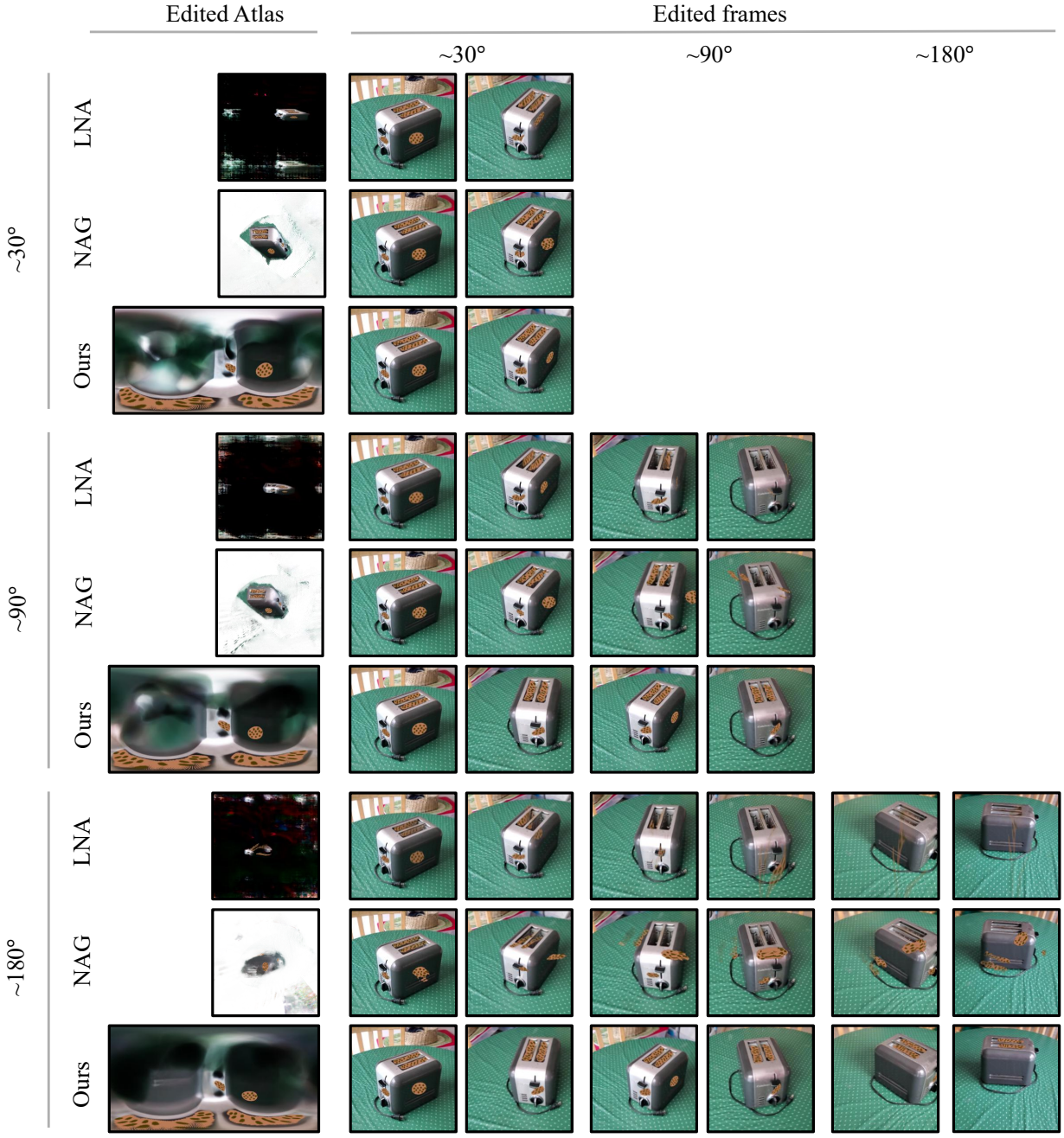



Figure 4. **Qualitative comparison on the** Toaster **sequence.** Following the same visualization protocol as Fig. 2, we compare canonical atlas reconstruction and consistent appearance editing using input subsequences covering $30^\circ$, $90^\circ$, and $180^\circ$ of object rotation. The reconstructed canonical atlases and selected edited frames are shown on the right.

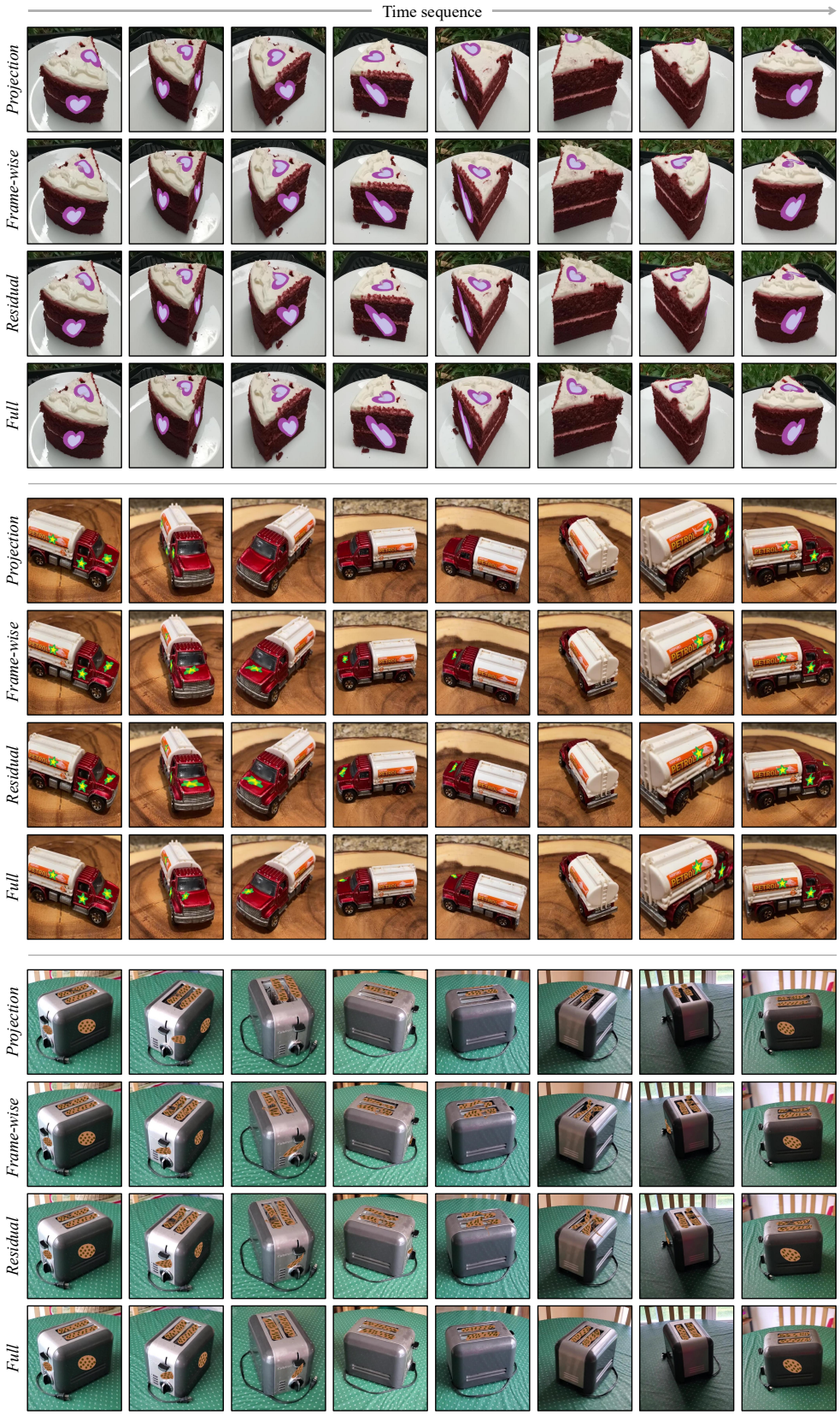



Figure 5. **Canonicalization ablation and edit propagation.** Qualitative comparison of the *Projection*, *Frame-wise*, *Residual*, and *Full* variants across the Cake, Toytruck, and Toaster sequences.

## References


[1] Adam W Harley, Yang You, Xinglong Sun, Yang Zheng, Nikhil Raghuraman, Yunqi Gu, Sheldon Liang, Wen-Hsuan Chu, Achal Dave, Suya You, et al. Alltracker: Efficient dense point tracking at high resolution. pages 5253–5262, 2025. 1

[2] Yoni Kasten, Dolev Ofri, Oliver Wang, and Tali Dekel. Layered neural atlases for consistent video editing. *ACM Trans. Gr.*, 40(6):1–12, 2021. 4

[3] Thomas Müller. tiny-cuda-nn, 2021. 3

[4] Jeremy Reizenstein, Roman Shapovalov, Philipp Henzler, Luca Sbordone, Patrick Labatut, and David Novotny. Common objects in 3d: Large-scale learning and evaluation of real-life 3d category reconstruction. pages 10881–10891, 2021. 3

[5] Jan Philipp Schneider, Pratik S Bisht, Ilya Chugunov, Andreas Kolb, Michael Moeller, and Felix Heide. Neural atlas graphs for dynamic scene decomposition and editing. 38: 14481–14500, 2025. 4

[6] Olga Sorkine and Marc Alexa. As-rigid-as-possible surface modeling. In *Proceedings of the Fifth Eurographics Symposium on Geometry Processing*, page 109–116, 2007. 5

[7] Pei Sun, Henrik Kretzschmar, Xerxes Dotiwalla, Aurelien Chouard, Vijaysai Patnaik, Paul Tsui, James Guo, Yin Zhou, Yuning Chai, Benjamin Caine, et al. Scalability in perception for autonomous driving: Waymo open dataset. pages 2443–2451, 2020. 4